

Operating System : HW1 – SiSH

32212859 윤덕용

남은 Freeday : 5

목차

1. Shell 이란 무엇인가?
2. Shell의 작동 원리
3. 코드 설명
4. 실제 작동
5. AI 사용

Shell 이란?

셸이란 운영체제에서 사용자와 커널 사이에서 사용자의 명령어를 커널이 이해할 수 있도록 해석해주는 역할을 한다. 사용자로부터 명령어를 입력 받으며, 이를 해석하여 커널에게 작업을 요청한다. 이후 작업이 완료되면 결과를 다시 출력해주는 역할을 맡으며, 환경변수 관리 또한 맡고 있다.

Shell의 작동 원리

셸에서 작동하는 여러 가지 명령어들은 모두 특정 경로에 저장되어 있으며, 셸은 입력 받은 명령어가 존재하는지 확인하기 위해 PATH 환경 변수를 순차적으로 탐색하여 명령어 이름과 일치하는 파일을 찾는다. 그리고 명령어가 내장 명령어일 경우, 부모 프로세스에서 실행하며, 외부 명령어일 경우 셸의 자식 프로세스를 생성하여 실행한다. 예를 들어 셸의 종료, 디렉토리 이동 같은 경우에는 셸 자체의 상태를 변경하여야 하므로 부모 프로세스에서 실행해야 한다. 따라서 이를 내장 명령어로 분류하며 반대로 외부 명령어인 ls, mkdir 등은 셸의 상태는 유지하되 해당 명령어의 작업은 수행해야 하므로 자식 프로세스를 만들어 실행하여 셸의 상태를 유지하며 작업을 수행한다고 할 수 있다.

코드 설명

```
int main(){
    char input[256];
    char input_copy[256];
    char cwd[1024];
    while(1){
        if(getcwd(cwd, sizeof(cwd)) == NULL){
            perror("getcwd failed");
            printf("SiSH>> ");
        }else{
            printf("SiSH >> %s >> ", cwd);
        }

        fgets(input, sizeof(input), stdin);
        input[strcspn(input, "\n")] = 0;

        if(strlen(input) == 0){
            continue;
        }
    }
}
```

셸의 기초 구성은 다음과 같다. Input 배열에 사용자 입력을 저장하며, 사용자 입력을 공백 단위로 나누기 위해 input_copy를 사용하여 입력을 복사해둔다. 또한 cwd 배열을 이용하여 현재 디렉토리를 표기하기 위하여 getcwd 함수를 이용하여 현재 디렉토리를 가져온다.

fgets()를 통해 사용자 입력을 받으며, 개행문자를 제거해주고 만약 빈 문자열이 입력됐을 경우 일반적인 셸과 마찬가지로 다시 입력하도록 유도한다.

```
strcpy(input_copy, input);
int i = 0;
char *token = strtok(input_copy, " ");
char *args[128];
while(token != NULL){
    args[i++] = token;
    token = strtok(NULL, " ");
}
args[i] = NULL;
char *command = args[0];

if(execute_builtin(command, args) == 1){
    continue;
}
```

다음으로 input을 이후에 써야하는 경우를 대비하여 입력값인 input을 input_copy에 복사해준다. Strtok()의 경우, 원본 문자열을 변경시키기 때문에 input을 바로 사용해선 안된다. 따라서 strtok로 공백을 기준으로 나누어 명령어와 arguments로 구분하여 저장한 후 마지막 argument 다음 배열의 값을 NULL로 명시하여 오류가 없도록 한다. 또한 *command 에 명령어를 저장함으로써 가독성을 높였다.

```
int execute_builtin(char *command, char *args[]){
    static char last_dir[1024];
    char current_dir[1024];
    if(getcwd(current_dir, sizeof(current_dir)) == NULL){
        perror("getcwd");
        return 1;
    }
    if(strcmp(command, "quit") == 0){
        printf("SiSH를 종료합니다.\n");
        fflush(stdout);
        exit(0);
    }
    if(strcmp(command, "cd") == 0){
        char *target_dir = args[1];
        if(target_dir == NULL || strcmp(target_dir, "~") == 0){ // cd 이후 인자가 없거나 home을 뜻하는 ~ 입력시 홈 디렉토리로 이동
            target_dir = getenv("HOME");
            if(target_dir == NULL){
                fprintf(stderr, "SiSH: cd: HOME 환경 변수를 찾을 수 없습니다.\n");
                return 1;
            }
        }else if(target_dir != NULL && strcmp(target_dir, "-") == 0){ // 이전 디렉토리로 이동
            if(last_dir[0] == '\0'){
                fprintf(stderr, "SiSH: cd: 이전 디렉토리 기록이 없습니다.\n");
                return 1;
            }
            target_dir = last_dir;
            printf("%s\n", target_dir);
        }
        if(chdir(target_dir) == 0){
            strcpy(last_dir, current_dir);
        }else{
            perror("SiSH: cd");
        }
        return 1;
    }
    return 0;
}
```

조건문 내에 존재하는 execute_builtin() 함수는 다음과 같다. cd, quit 의 경우, 외부 명령어가 아닌 내장 명령어이므로 함수로 분리하여 작성하였다. 입력된 명령어를 strcmp로 비교하여 cd, quit 등을 실행하도록 구성하였으며, cd의 경우 뒤의 argument에 따라 존재하지 않거나 ~과 같은 홈 디렉토리를 의미하는 경우 홈 디렉토리로 이동할 수 있도록 하였으며, 함수 내에 static 변수를 선언하여 cd를 통해 디렉토리를 이동할 때 마다 last_dir에 디렉토리를 저장하여 cd - 를 입력할 시 이전 디렉토리로 이동할 수 있도록 구성하였다. 입력된 명령어가 내장 명령어인 경우 1을 반환하여 다시 루프를 시작하도록 하였다.

```

char *path_env = getenv("PATH");
char full_path[1024];
int command_found = 0;

if(path_env != NULL){
    char path_copy[1024];
    strcpy(path_copy, path_env);
    char *dir = strtok(path_copy, ":");

    while(dir != NULL){
        sprintf(full_path, "%s/%s", dir, command);
        if(access(full_path, F_OK) == 0){
            command_found = 1;
            break;
        }
        dir = strtok(NULL, ":");
    }
}

```

내장 명령어가 아닌 경우, 외부 명령어 혹은 정의되지 않은 명령어이므로, 외부 명령어의 환경변수가 필요하므로 `getenv("PATH")`를 통해 경로를 `*path_env`에 저장하였다. 이전 input과 마찬가지로 원본 경로가 바뀌지 않도록 복사해준 뒤 `:`로 구분 되어있는 경로들을 나누어 분리하였으며, `dir`에 저장된 경로를 경로/명령어로 설정한 후 `F_OK`를 통해 해당 파일이 존재하는지 확인하여 존재하는 경우 `command_found = 1`로 설정하여 이후 외부 명령어 실행 과정인 `fork()`를 실행할 수 있도록 구성하였다.

```

        if(command_found){
            pid_t pid = fork();
            if(pid == 0){ // 자식프로세스 실행
                execve(full_path, args, environ);
                perror("execve failed");
                exit(EXIT_FAILURE);
            }else if(pid > 0){ // 부모프로세스 실행
                int status;
                waitpid(pid, &status, 0);
            }else{
                fprintf(stderr, "SiSH: fork failed %s\n", strerror(errno));
            }
        }else{
            fprintf(stderr, "SiSH: command not found: %s\n", command);
        }
    }
    return 0;
}

```

Command_found가 1인 경우 (명령어가 존재할 경우) `fork()`를 통해 자식 프로세스를 실행

행하여 자식프로세스에서 외부명령어를 실행하도록 한 후 종료한다. 그 동안 부모 프로세스는 waitpid()를 통해 자식 프로세스가 종료될 때 까지 대기하며, 만약 command_found가 1이 아닌경우, 존재하지 않는 명령어로 판단하여 에러 메시지를 출력하고 루프를 다시 시작한다.

실제 작동

```
SiSH >> /home/ydy236/2025_os_hw1/2025_os_hw1 >> cd
SiSH >> /home/ydy236 >> cd -
/home/ydy236/2025_os_hw1/2025_os_hw1
SiSH >> /home/ydy236/2025_os_hw1/2025_os_hw1 >> ls
a.out fork2.c fork.c getenv.c LICENSE README.md sish.c stat.c test
SiSH >> /home/ydy236/2025_os_hw1/2025_os_hw1 >> ls -l
total 56
-rwxrwxr-x 1 ydy236 ydy236 17176  9월 30 15:53 a.out
-rw-rw-r-- 1 ydy236 ydy236  1842  9월 23 22:18 fork2.c
-rw-rw-r-- 1 ydy236 ydy236   456  9월 23 22:18 fork.c
-rw-rw-r-- 1 ydy236 ydy236   551  9월 23 22:18 getenv.c
-rw-rw-r-- 1 ydy236 ydy236  1078  9월 23 22:18 LICENSE
-rw-rw-r-- 1 ydy236 ydy236  4494  9월 23 22:18 README.md
-rw-rw-r-- 1 ydy236 ydy236  2703  9월 30 15:52 sish.c
-rw-rw-r-- 1 ydy236 ydy236   402  9월 23 22:18 stat.c
drwxrwxr-x 2 ydy236 ydy236  4096  9월 27 16:04 test
```

```
SiSH >> /home/ydy236/2025_os_hw1/2025_os_hw1 >> cat fork.c
#include <stdio.h>
#include <unistd.h>
#include <errno.h>
#include <sys/types.h>
#include <sys/wait.h>
#include <stdlib.h>

int main(int argc, char *argv[])
{
    pid_t pid;
    int i;

    for (i = 0 ; i < 10 ; i++) {
        pid = fork();
        if (pid == -1) {
            perror("fork error");
            return 0;
        }
        else if (pid == 0) {
            // child
            printf("child process with pid %d (i: %d) \n", getpid(),
i);
            exit (0);
        } else {
            // parent
            wait(0);
        }
    }
    return 0;
}

SiSH >> /home/ydy236/2025_os_hw1/2025_os_hw1 >> pwd
/home/ydy236/2025_os_hw1/2025_os_hw1
SiSH >> /home/ydy236/2025_os_hw1/2025_os_hw1 >> █
```

```
SiSH >> /home/ydy236/2025_os_hw1/2025_os_hw1 >> quit  
SiSH를 종료합니다.  
ydy236@assam:~/2025_os_hw1/2025_os_hw1$
```

SiSH가 작동할 때는 SiSH >> 현재 디렉토리 >> 입력 의 형식으로 작동하며, cd, quit 등 직접 정의한 내장 명령어와 ls, cat 등 외부 명령어 모두 작동하는 모습을 볼 수 있다.

AI 사용

**environ, access(), getenv(), getcwd()와 같은 명령어들의 정의 및 사용법에 대해 AI 툴을 사용하여 숙지하고 사용하였습니다. 또한 환경변수의 개념 및 사용법에 대해 잘 몰랐기 때문에 AI를 사용하여 사용법을 익혔습니다. 또한 Makefile 관련 지식에 대해 AI를 통해 알게 되었습니다.